FOL: Bridging CLOS and Clojure via the Metaobject Protocol

Frank Adrian
Ancar Technology
Olathe, KS, USA
frank.adrian314159@gmail.com

Abstract

FOL (Functional Object Lisp) provides CLOS-style object orientation backed by persistent data structures, using the Common Lisp Metaobject Protocol (MOP). We contribute: (1) a persistent metaclass system with hybrid storage and lazy schema migration, and (2) a specificity-ordered predicate dispatch model. Persistent slot storage incurs 1.4–2.1× read overhead. In an AST optimization engine with 25 node classes and 54 predicate-dispatch rewrite rules, FOL is 19.2× slower than a `defstruct+typecase` baseline while providing $O(1)$ speculative rollback. Whole-program overhead in a 1.18-billion-gate event-queue simulation narrows to 6.9×. Our predicate dispatch introduces a syntactically computable, level-based ordering that provides a predictable alternative to logical subsumption.

CCS Concepts

• **Software and its engineering → Functional languages**; **Object oriented languages**; *Extensible languages.*

Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP

FOL (Functional Object Lisp) synthesizes CLOS with persistent data structures. By leveraging the MOP [9], FOL provides intrinsic object persistence, lazy schema migration, and a novel predicate dispatch system.

While libraries like FSet provide functional collections, FOL unifies these paradigms by representing objects as versioned points in time, where "mutation" is reified as a transition between immutable snapshots.

FOL addresses the "new operations" dimension of the Expression Problem: adding a traversal such as `count-ops`—which recurses over `:left`/`:right` slots and returns 0 at leaves—requires only a new `defmethod` with one clause per handled node type; no existing class definitions change. After rolling back a failed optimization pass, `count-ops` applies to the reverted tree without copying. Like CLOS, adding a new node type still requires a new clause in each open generic that must handle it; the FOL advantage is that all existing clauses remain correct over any mix of original and speculatively-modified nodes. Automated schema evolution makes versioning a first-class concern (Section 3.2).

FOL adopts Clojure's literal syntax and core sequence API. Following Hickey's epochal time model [7], it distinguishes *identity* (a mutable reference box like an `Atom`) from *value* (the progression of versions).

1 Architecture

1.1 Object Implementation

Slot storage uses a hybrid strategy: objects with ≤ 8 slots use native CLOS slots, while wider objects overflow into a 32-way persistent trie. This threshold keeps ≈80% of classes [11] on the native fast path (1.4 − 2.1× mutable-CLOS read). Redefinitions crossing this threshold are handled lazily at the next `assoc`.

Immutability is enforced via the `persistent-class` metaclass, which intercepts slot access (Table 1) to: (1) guard against mutation; (2) map logical slots to the hybrid layout; and (3) transform instances for schema evolution. A reserved native slot `%metadata` stores the object's metadata map; it is excluded from structural equality via `=` to maintain value-identity stability across operational updates—attaching a `:last-accessed` timestamp to an object, for instance, does not alter its identity as a key in a persistent map or invalidate a prior hash-join result.

1.2 Collections and Runtime

FOL implements persistent **Vectors** (32-way tries) [12], **Maps/Sets** (HAMTs) [2], and **Sorted collections** (32-way B-trees). A branching factor of 32 ensures depth ≤ 5 for million-element collections, bounding path-copying costs. For keyword keys, HAMT lookup is ≈1.3× faster than standard CL `gethash` on SBCL, as EQL-based key comparison avoids the general hashing path and index computation uses POPCNT. Transducers (Section 2) provide a unified API. FOL transpiles to Common Lisp, inheriting its generational GC performance.

Space complexity. FOL does not retain historical versions—old snapshots become eligible for GC as soon as no live reference remains. Structural sharing bounds the incremental cost of each `assoc` to $O(\log_{32} N)$ new nodes (one path copy), not $O(N)$. Short-lived intermediates—loop variables, traversal temporaries—are reclaimed in SBCL's nursery, keeping GC overhead at 2.0% in the 1.18B-gate LSim experiment.

2 Features

FOL implements **Transducers** [8], **Lazy sequences**, **Atoms** for state management, and Clojure-style **Macros**. **Transient builders** reduce per-write allocation overhead for bulk updates by confining mutation to a thread-local window; W successive writes cost $O(W \log_{32} N)$ with no intermediate path-copy allocation.

Generic functions (GFs) dispatch by arity, then specificity. **Closed dispatch** (`defn`) emits a `cond` cascade, avoiding dynamic scans. **Open dispatch** (`defmethod`) supports CLOS-style extensibility.

Figure 1 gives the formal grammar for FOL's multi-pattern dispatch syntax.

Table 1: MOP Protocol Adaptation

Protocol	Status	Rationale
`slot-value-using-class`	Redirected	Reads from base object or overflow trie
`(setf slot-value-using-class)`	Guarded	Blocks post-init writes
`slot-boundp-using-class`	Redirected	Checks object slot membership
`slot-makunbound-using-class`	Blocked	Immutability invariant — use `dissoc` to unbind slots
`validate-superclass`	Extended	Cross-metaclass mixing
`initialize-instance`	Extended	Populates base object and overflow trie from initargs; stamps `%schema-version` at birth
`reinitialize-instance`	Extended (`:before` on metaclass)	Snapshots current alias-map, overflow indices, and version counter into a `class-version` struct before each redefinition; advances the version counter
`compute-slots`	Specialized	Implements hybrid native/trie layout
`update-instance-for-redefined-class`	Implemented (`:after`)	Multi-hop chain replay: composes alias-maps from version chain back to birth version, recovering renamed values (Sec. 3.2)
`change-class`	Omitted	Violates immutability; structural transformations beyond renaming must specialize `update-instance-for-redefined-class`

Method combination uses standard CLOS execution semantics (`:before`/`:after`/`:around`), with method ordering determined by $\prec_{disp}$ in place of the class p

defn	::=	`(defn` *name* (*s-clause* | *m-clause*)) | `(fn` (*s-clause* | *m-clause*))
defg/m	::=	`(defgeneric` *name*`...)` | `(defmethod` *name qual*? (*s-clause* | *m-clause*))
s-clause	::=	`[`*param**`]` *body*+
m-clause	::=	`{(`*symbol param* | `:keys [(`*symbol* | `(`*sym param*`))`*`])`*`}` *body*+
param	::=	*symbol* | *type* | `(`*symbol type*`)` | `(`*symbol* `(`*fn arg**`))` | *destr*
destr	::=	`[`*param**`]` | `{:keys [(`*symbol* | `(`*sym param*`))`*`]}`
type	::=	<*type-name*> | `:before` | `:after` | `:around`

Figure 1: FOL Multi-Pattern Dispatch Grammar (compressed)

3 Predicate Dispatch

3.1 Dispatch Ordering Rules

FOL extends pattern matching with predicate specializers (`var (fn args...)`). Specificity categories (Table 2) order patterns by constraint strength (Level 3 > 2 > 1 > 0), resolving same-level ties via definition order. This *syntactic precedence model* selects the narrowly specialized form (e.g., `even?` over `<integer>`) via a computable total order $\prec_{\text{disp}}$. Within each level, the ordering mirrors semantic subsumption: types via the class hierarchy DAG, structural patterns via argument-arity containment, wildcards by definition. Cross-level priority (predicate over type over destructuring over wildcard) is a design choice, not a semantic derivation. Level 3 (arbitrary predicates) is ordered by definition index only; the system detects no semantic subsumption (e.g., `prime?` vs `pos-int?`) within this stratum. In practice, programmers cannot declaratively express that one independent dynamic predicate subsumes another; they must handle arbitrary overlaps by load-order conventions or encapsulate them in a single clause. This is a deliberate design tradeoff: sacrificing theorem-prover exactness for an algorithmically bounded compilation phase. All method qualifiers (`:before`, `:after`, `:around`) follow this computed order. Standard CLOS execution semantics are preserved—e.g., `:before` methods run most-specific-first, `:after` run least-specific-first—but the sequence they iterate over is $\prec_{\text{disp}}$ instead of the class precedence list.

Table 2: Predicate Specificity Categories

Level	Category	Example
3	Predicates	`(x (even?))`
2	Types	`(x <integer>)`
1	Destructuring	`[x y]`
0	Wildcard	`x`

For compound predicates, the level calculus applies $\mathcal{L}(\text{and } p_1 \ldots p_n) = \max_i \mathcal{L}(p_i)$: a conjunction of type predicates is Level 2 (static), while a mixed conjunction is Level 3 (dynamic). This is semantically conservative but sound.

Comparison to `defmulti`. Clojure's `defmulti` [6] dispatches on the *return value* of a user-supplied dispatch function applied to the arguments; ordering ambiguities are resolved manually via `prefer-method` and `derive`-based hierarchy declarations. This is flexible but requires the programmer to maintain preference declarations whenever two methods could match the same call. FOL's

syntactic total order $\prec_{\text{disp}}$ eliminates this burden for the common case: patterns differing in type or structural depth are automatically ordered without annotation. The residual hazard—incomparable Level-3 predicates, discussed in Section 5—has the same scope as Clojure's `prefer-method` requirement, but is narrower because structural and type differences are already resolved syntactically.

3.1.1 Formal Specificity Ordering. We formalize the specificity relation $\prec$ using the inference rules summarized in Table 3. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern p.

Table 3: Specificity Inference Rules

Rule	Formalization
Spec-Level	$\dfrac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2}$
Spec-Prefix	$\dfrac{p_1 \prec q_1}{[p_1, \dots] \prec [q_1, \dots]}$
Spec-Tail	$\dfrac{\mathcal{L}(p_1) = \mathcal{L}(q_1) \qquad [p_2, \dots] \prec [q_2, \dots]}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n]}$
Spec-Length	$\dfrac{\mathcal{L}(p_i) = \mathcal{L}(q_i)\ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n, p_{n+1}] \prec [q_1, \dots, q_n]}$
Spec-Fixed	$\dfrac{\mathcal{L}(p_i) = \mathcal{L}(q_i)\ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n\ \&\ r]}$
Spec-Map-Prefix	$\dfrac{p_1 \prec q_1}{\{k_1\ p_1, \dots\} \prec \{k_1\ q_1, \dots\}}$
Spec-Map-Tail	$\dfrac{p_1 = q_1 \qquad \{k_2 \dots\} \prec \{k_2 \dots\}}{\{k_1\ p_1, k_2\ p_2, \dots\} \prec \{k_1\ q_1, k_2\ q_2, \dots\}}$
Spec-Keys	$\dfrac{\text{keys}(q) \subset \text{keys}(p)}{p \prec q}$

The operational dispatch order $\prec_{\text{disp}}$ resolves ties using definition order. Let $idx(p)$ denote the definition index of the method clause containing p.

$$p_1 \prec_{\text{disp}} p_2 \iff p_1 \prec p_2 \vee (\neg(p_2 \prec p_1) \wedge idx(p_1) < idx(p_2))$$

Patterns of different arity are incomparable; dispatch selects the arity group first, then applies total order $\prec_{\text{disp}}$. Lexicographic levels ensure $p \prec_{\text{disp}} q$ or $q \prec_{\text{disp}} p$ for all same-arity patterns. While definition-order tie-breaking is a modularity hazard, it follows Lisp's interactive workflow.

Lemma 3.1 (Totality). *$\prec_{disp}$ is a strict total order on same-arity patterns.*

Proof. We first define $\prec$ and show it is a strict partial order (irreflexive and transitive). Irreflexivity follows from the strict inequality of levels or lengths in the base cases. Transitivity holds because the inference rules effectively encode $\prec$ as a lexicographic comparison on a canonical integer tuple (e.g., level-vector, key-count), preserving the standard transitivity of tuple ordering.

Totality of $\prec_{\text{disp}}$ is then established by augmenting $\prec$ with the definition index *idx*:

(1) **Lexicographic dominance (sequential patterns):** Each position p_i carries a level $\mathcal{L}(p_i) \in \{0, 1, 2, 3\}$. Spec-Level, Spec-Prefix, and Spec-Tail together impose a left-to-right lexicographic order on level vectors. When levels tie, Spec-Length and Spec-Fixed resolve length differences.
(2) **Structural incomparability (cross-form patterns):** Map and sequential patterns of the same arity are structurally incomparable under $\prec$: $\neg(p \prec q) \wedge \neg(q \prec p)$. Similarly, non-nested map patterns with disjoint or incomparable key sets are incomparable.
(3) **Map consistency:** For map patterns with the same key set, entries are walked in alphabetical key order, applying $\prec$ recursively.

Because every pair (p, q) is either ordered by $\prec$, $q \prec p$, or resolves to *idx* (which is a total order on the finite set of clauses), $\prec_{\text{disp}}$ is a strict total order. □

Example: nested tie-breaking. To illustrate how the rules compose through nested structures, consider two clauses of a generic function `handle-event` that dispatches on a single map argument containing a `:data` key whose value is a two-element vector:

p (index 10): `[{:keys [(data [x <integer>])]}]`
q (index 11): `[{:keys [(data [x y])]}]`

Both are single-parameter clauses whose top-level parameter is a map destructure, so $\mathcal{L}(p_1) = \mathcal{L}(q_1) = 1$ (Level 1, Destructuring). The top-level levels tie; the system enters the structure via Spec-Map. Both patterns share the same key set `{:data}`, so Spec-Keys produces no ordering and Spec-Map compares the value bound to `:data` in each pattern: `[x <integer>]` versus `[x y]`. These are both Level-1 sequential patterns, so Spec-Tail walks the positions lexicographically: position 1 ties ($\mathcal{L}(x) = \mathcal{L}(x) = 0$); at position 2, $\mathcal{L}(\texttt{<integer>}) = 2$ (Type) versus $\mathcal{L}(y) = 0$ (Wildcard). Spec-Level fires: $2 > 0$, establishing `[x <integer>]` $\prec$ `[x y]`, and therefore $p \prec q$ by Spec-Map. At runtime, p is tried first; a non-integer second element falls through to q.

3.1.2 Dispatch Performance. Predicate dispatch requires an $O(N)$ linear scan per call, as Level-3 resolution cannot be statically cached. Methods are sorted once at load time in $O(N \log N)$ using the syntactic level order; theorem-prover-based systems instead require pairwise subsumption checks at load time (at least $O(N^2)$ per generic function) but produce faster cached dispatch. To mitigate runtime cost, FOL partitions methods by arity group, ensuring calls only scan matching-arity methods. `call-next-method` within specialized auxiliaries resolves the next most specific method under $\prec_{\text{disp}}$.

3.1.3 Transpiler Architecture. FOL is implemented as a transpiler targeting Common Lisp. The compilation pipeline operates in four stages. First, **Reader expansion** converts FOL literal syntax into standard Lisp forms. Second, **Macro expansion** resolves Clojure-style macros into core FOL primitives. Third, **Code generation** emits Common Lisp code, wrapping slot access and method definitions with MOP-integrated metadata. Finally, **Linking** loads the generated code into the host environment.

Listing 1 illustrates a two-clause `defn` with a type specializer. The compiler emits a direct `defun` containing a specificity-ordered

cond cascade; no runtime dispatch infrastructure is invoked, as the ordering is resolved statically at compile time.

Listing 1: Transpilation: specificity-ordered multi-clause defn

```
;; FOL source: type specializer (Level 2) before wildcard (Level 0)
(defn classify
  [(x <integer>) :integer]
  [x             :other])

;; Generated Common Lisp: ordering preserved as cond cascade
(defun classify (x)
  (cond
    ((typep x (find-class '<integer>)) :integer)
    (t                                 :other)))
```

3.1.4 Usage Patterns. **Structural Diff** (Listing 2) uses an :around method on assoc to detect changes by comparing snapshots, triggering automatic change tracking. **Speculative Execution** (Listing 3) enables rejecting updates by returning the pre-update snapshot (obj), which in mutable CLOS would require an explicit pre-call copy or undo log. This pattern provides $O(1)$ single-object rollback: the pre-update snapshot is an ordinary value on the stack, returned instead of the modified object with no copying or undo logging required. **Content routing** via predicate dispatch (Listing 4) allows sensors to trigger alerts based on data values without manual cond cascades.

Listing 2: Automatic Structural Diff

```
(defmethod assoc :around [(obj <diffable>) key val]
  (if (= key :_changes)
      (call-next-method)          ; avoid re-entrant tracking
      (bind [old (get obj key)
             new-obj (call-next-method)]
        (if (= old (get new-obj key))
            new-obj
            (assoc new-obj :_changes (inc (get obj :_changes 0)))))))
```

Listing 3: Speculative Execution Guard

```
(defmethod assoc :around [(obj <guarded>) key val]
  (if (valid-update? obj key val)
      (call-next-method)
      obj))  ; discard update by returning original
```

Listing 4: Value-Based Alerting

```
(defmethod notify [{:keys [(temp (>= 100))]}]
  (print "ALERT: Critical Temperature!"))
```

3.1.5 Persistence and Versioning Performance. Versioning overhead—the cost of recovering the current state of an object via chain-replay—is $O(H)$ where H is the number of redefinitions. Since redefinitions are infrequent, H is typically small (< 10). Replay cost is bounded by a few microseconds, paid once per dormant instance on first access after redefinition. Once migrated, the object's slots reside in the current schema's preferred layout (native or trie), and subsequent reads proceed at the standard $1.4 - 2.1\times$ read rate.

3.2 Schema Evolution

The :after method on update-instance-for-redefined-class (*chain replay*) walks a *version chain* on the metaclass, composing single-hop alias maps from Birth to Current. This map is consulted against the property-list of discarded slots to recover renamed values, rebuilding the overflow trie if necessary. This chain migration is structurally confluent (order-independent) as it accumulates key renames. To guarantee execution safety, the macro-expansion step detects cyclical renames (e.g., $A \to B \to A$) and transitive conflicts (both X and Y renamed to Z), signaling a translation error before the class runtime is redefined.

Listing 5: Lazy Schema Migration with Aliasing

```
;; Schema 1
(defclass <sensor> [] [[id] [reading]])
;; Schema 2: rename reading -> val; :reading keyword still routes to
       val
(defclass <sensor> [] [[id] [val :alias reading]])
```

4 Evaluation

4.1 Common Lisp and Clojure Comparison

FOL's novel mix of persistent objects and specificity-ordered predicate dispatch fills a gap between CLOS and Clojure (Table 4). While Clojure's defrecord provides map-backed values, FOL provides MOP-integrated persistence with full CLOS method combinations and MOP extensibility.

Table 4: Built-in Feature Comparison

Feature	FOL	Clojure	CL
Static type safety	×	×	×
Persistent objects	✓[d]	○[c]	×
CLOS/MOP	✓	×	✓
Method combinations	✓	×	✓
Predicate dispatch	✓	single[a]	×[b]
Transducers	✓	✓	×
Macros	✓	✓	✓
Raw scalar performance	Low	High	High

[a] defmulti dispatches via a user-supplied function (arbitrary arity); prefer-method and derive-based hierarchies allow manual ordering overrides, but there is no automatic multi-parameter specificity ordering derived from the syntactic form of the dispatch expression.
[b] Multi-parameter type dispatch natively; libraries like trivia add pattern matching.
[c] Clojure has persistent *data structures*; defrecord produces a map-backed value type, not a MOP-integrated persistent object.
[d] Metaclass-integrated persistent class instances: slot storage is backed by immutable HAMTs, (setf slot-value) is blocked, and functional updates are performed via assoc. Persistence is enforced transparently by the metaclass protocol.

Functional updates are 33–447× slower than mutation. Naive Quicksort and BFS (Listings 6–7) illustrate the path-copying penalty (Table 5), with Quicksort 100k incurring 90× overhead. Transients recover performance by batching these updates.

Table 5 shows results for all three implementations on both workloads (SBCL 2.6.0, AMD Ryzen 9 5900X, 3 runs each).

Quicksort overhead is dominated by $3.4M$ path copies (90×). Transient Quicksort reduces this to 39× and cuts allocation by 97%. BFS shows similar gains (35× → 6.5×). These results establish a translation penalty; idiomatic reformulations avoid this by design (Table 6).

Listing 6: Naive Persistent Partition

```
(defn partition [v low high]
  (bind [pivot (get v high)]
    (loop [j low i (dec low) curr-v v]
      (if (< j high)
          (if (<= (get curr-v j) pivot)
              (bind [next-i (inc i)
                     v1 (assoc curr-v next-i (get curr-v j))
                     v2 (assoc v1 j (get curr-v next-i))]
                (recur (inc j) next-i v2))
              (recur (inc j) i curr-v))
          (bind [next-i (inc i)
                 v1 (assoc curr-v next-i (get curr-v high))
                 v2 (assoc v1 high (get curr-v next-i))]
            [v2 next-i])))))
```

Listing 7: Naive Persistent BFS Step

```
(defn bfs-step [q dists graph]
  (if (empty? q)
      dists
      (bind [u (first q)
             edges (get graph u)
             d (get dists u)]
        (loop [es edges nq (rest q) nd dists]
          (if (empty? es)
              (bfs-step nq nd graph)
              (bind [v (first es)]
                (if (= (get nd v) -1)
                    (recur (rest es)
                           (conj nq v)
                           (assoc nd v (inc d)))
                    (recur (rest es) nq nd))))))))
```

Table 5: Adversarial Workload Results

Workload	Implementation	Time	Alloc	vs. CL
QuickSort	CL (simple-vector)	30 ms	0.8 MB	1.0×
$N = 100{,}000$	FOL Persistent (assoc)	2745 ms	2947 MB	~90×
	FOL Transient (assoc!)	1165 ms	91 MB	~39×
BFS	CL (hash-table)	8 ms	2.3 MB	1.0×
$N = 50{,}000$	FOL Persistent (assoc)	290 ms	119 MB	35×
	FOL Transient (assoc!)	52 ms	13 MB	6.5×

Idiomatic merge sort recurses on non-overlapping halves via `subvec` and builds each merged result by successive `conj` calls—no per-element swaps. **BFS with lazy distance initialization** starts from an empty persistent dict and adds each node only when first discovered, eliminating the N up-front `assoc` calls of the naive version. Each algorithm was measured in two FOL variants—persistent and transient—to assess where the transient-builder protocol is beneficial. Table 6 shows the results.

Merge sort overhead (34×) stems from $O(\log N)$ trie node touches ($1.7M$ calls). Lazy BFS (28×) improves on naive BFS by eliminating initialization waste. Transients help BFS (28× → 13×) but are counterproductive for Merge Sort (34× → 37×): persistent vector `conj` already touches only a root-to-leaf path of depth ≤ 5 per append, so transient setup overhead is not recovered—it actively adds cost at this scale.

Table 6: Idiomatic Algorithm Results: persistent and transient FOL vs. native CL

Workload	Implementation	Time	Alloc	vs. CL
Merge Sort	CL (simple-vector)	32 ms	1.6 MB	1.0×
$N = 100{,}000$	FOL persistent (`conj`)	1080 ms	623 MB	34×
	FOL transient (HAMT array)	1190 ms	81 MB	37×
BFS lazy	CL (hash-table)	7 ms	2.3 MB	1.0×
$N = 50{,}000$	FOL persistent (lazy dict)	200 ms	61 MB	28×
	FOL transient (`hamt-assoc!`)	94 ms	16 MB	13×

The bottleneck is memory pressure from intermediate allocations, not complexity. Transients fuse updates, bringing bulk costs closer to mutable equivalents where many updates occur within a transient scope.

4.2 Feature Benchmarks

Table 7: Feature Benchmark Summary

Benchmark	Perf Ratio	Mem Ratio
Structural Diff[a]	113.7×	55.3×
Speculative Execution	23.4×	9.0×
Observable Slots	6.69×	6.32×

[a] Using six sequential `assoc` calls per iteration.

Table 7 summarizes overheads. CL baselines use direct `setf`-based mutation: a manually-maintained change counter for Structural Diff, a standard `defmethod` with explicit value checks for Observable Slots, and a guard method with a pre-call object copy for Speculative Execution. These feature benchmarks represent worst-case scenarios, isolating the bottleneck operation without amortizing its cost over a larger workload; in aggregate workloads like LSim, overhead narrows to 6.9×.

4.3 Abstract Syntax Tree Rewriting Engine

To evaluate persistent objects and predicate dispatch together, we implemented an AST optimization engine. The benchmark defines 25 persistent node classes: one abstract base `<ast-node>`, one `<op-var>` (a leaf), one `<op-lit>` (a literal with a `:val` slot), and 23 binary operator classes each with `:left` and `:right` slots. A single multi-clause `defmethod` encodes 54 algebraic rewrite rules via predicate dispatch, including nested structural patterns such as the identity-element rule:

```
(defmethod ast-optimize
  [({:keys [(left ({:keys [(val (eql 0))]} <op-lit>))
            (right r)]} <op-add>)]
  r)
```

A second 25-clause `walk` method drives recursive descent. The test tree is a depth-14 linear chain (29 nodes). The CL baseline uses `defstruct` for all node types and a `typecase` cascade for dispatch.

FOL is 19.2× slower and uses 7.2× more memory per optimization pass. The overhead has two sources. **Field reads** via the `get` generic function cost ≈50 ns versus ≈4 ns for a struct slot reference

Table 8: AST Optimizer: FOL vs. CL baseline (N = 10,000 optimizations).

Implementation	Time	Alloc	vs. CL
CL (`defstruct` + `typecase`)	5 ms	4.2 MB	1.0×
FOL (persistent + predicate dispatch)	92 ms	30.2 MB	19.2×

(12×). **Object construction** via `make-<class>` costs ≈183 ns versus ≈37 ns for mutable `make-instance` (5×); see Section 4.6 for the `%make-persistent` construction analysis.

To attribute the 19.2× total, the dispatch micro-benchmark (Section 4.6.1) shows `defgeneric` at ≈1.6× over CLOS type dispatch for pure Level-3 predicate evaluation against mutable objects; this is an upper bound on AST routing overhead, since the AST benchmark uses mixed-level dispatch (Level-2 type checks and Level-1 structural patterns that short-circuit before reaching Level-3 `eql` checks). In the AST workload, each `ast-optimize` clause that does fire evaluates its structural guard by reading persistent slots (e.g., checking `:left` and `:val`), so dispatch cost compounds with field-read cost rather than being independent. This compounding predicts ≈1.6× × 12× ≈ 19×, consistent with the observed 19.2×. A mutable CLOS visitor using standard type dispatch would incur only the routing overhead; the ≈12× residual is the cost of persistent field access. In exchange, speculative rewrite passes are $O(1)$ to roll back: discarding a modified tree releases the new root with no deep copy required.

4.4 Logic Simulation (LSim)

LSim is an event-driven digital logic simulator that exercises persistent data structures at scale. Circuits are composed of hierarchically-defined modules that are flattened into primitive NAND gates at simulation time. To isolate the cost of the event-queue data structure, we hold gate-connectivity representation constant across both variants and compare only the priority queue: the **CL** baseline uses a mutable destructive leftist heap [4] (in-place merge, $O(1)$ allocation per pop/push), while the **FOL** variant uses a persistent leftist heap [12] ($O(\log n)$ structural copies per merge). Table 9 presents the results across six configurations spanning five orders of magnitude in gate evaluations; the largest circuits were run with a 56 GB heap (`-dynamic-space-size 57344`). To independently attribute overhead between the queue and simulation-state data structures—relaxing the connectivity-constant constraint—we designed a 2×2 ablation study (Table 10).

The ratio peaks at 9.4× (8x32-900) and narrows to 6.9× at the largest configuration. Normalising by gate evaluations clarifies the mechanism: CL's per-gate simulation cost is flat at 1.4–3.3 μs/eval across all six configurations, because the mutable leftist heap's working set remains bounded by $O(\log n)$ cons cells on the merge path—small enough to stay cache-resident regardless of circuit size. FOL starts at ≈30 μs/eval for the smallest circuit and converges toward ≈12 μs/eval at the two largest as the fixed $O(\log n)$ per-merge allocation cost is amortized over more simulation work per queue operation. GC accounts for only 2.0% of FOL run time at large scale.

4.5 Spatial Locality and Cache Effects

Since gate-connectivity is held constant across both PQ variants, the locality difference is confined to the event queue. The mutable leftist heap reuses existing cons cells in-place: after a pop, the freed cells are reclaimed and reused in the next push, keeping the live working set bounded by $O(\log n)$ recently-touched cons cells—highly cache-hot. The persistent leftist heap allocates $O(\log n)$ fresh nodes per merge; these objects are newly promoted through the nursery generation and their addresses are not spatially correlated with older nodes. At small scales this allocation pressure dominates. As circuits grow, the ratio narrows because the amount of actual simulation work per queue operation increases proportionally to the number of active gates, diluting the fixed $O(\log n)$ allocation cost per merge; SBCL's generational GC handles the short-lived nodes with only 2.0% overhead at large scale, so the narrowing is driven by amortization of per-merge cost over more simulation work rather than by GC improvement.

Table 9: LSim PQ Scaling Results (mean CPU time).

Config	Gate Evals	CL	FOL	Ratio
8bit-100	876	2.9 ms	26.5 ms	9.1×
32bit-300	10 224	27.4 ms	254 ms	9.3×
8x32-900	281 248	559 ms	5.27 s	9.4×
32x32-3000	3 850 304	8.91 s	79.1 s	8.9×
8x32x32-9000	89 708 032	126 s	903 s	7.2×
32x32x32-30000	1 183 510 528	1 990 s	13 780 s	6.9×

Normalised by gate evaluations, FOL converges toward ≈12 μs/eval, confirming that persistent heap overhead is bounded by the fixed $O(\log n)$ per-merge cost, not by circuit size.

We designed a 2×2 ablation (Table 10) to attribute overhead between the event-queue and simulation-state.

Table 10: 2×2 Ablation: Mutable vs. Persistent Heap × State Maps (mean wall time; ratios relative to CL baseline).

Circuit	CL	Hybrid-A (persist. maps)	Hybrid-B (persist. heap)	FOL
32bit-300	11.8 ms	24.1 ms (2.0×)	42.0 ms (3.6×)	118 ms (10.0×)
8x32-900	319 ms	724 ms (2.3×)	953 ms (3.0×)	2.52 s (7.9×)
32x32-3000	3.62 s	9.24 s (2.6×)	9.70 s (2.7×)	33.8 s (9.4×)
8x32x32-9000	131 s	345 s (2.6×)	246 s (1.9×)	1,024 s (7.8×)

Persistent maps impose 2.0–2.6× slowdown, while **persistent heap** overhead decreases with size (3.6× → 1.9×) as circuit scale grows (Section 4.4). At the largest scale (8×32×32-9000), the combined cost is approximately explained by the product of both overheads scaled by the ≈1.6× dispatch factor (2.6 × 1.9 × 1.6 ≈ 7.9×, vs. observed 7.8×); at smaller scales the interaction is sub-multiplicative, likely because persistent-map and persistent-heap allocation pressure overlap in the same nursery generation rather than adding independently.

4.6 Micro-Benchmarks

We measured performance for persistent objects vs mutable CLOS across slot counts. **Construction time** for a 2-slot object is ≈183 ns vs ≈37 ns for mutable `make-instance` (5×) and ≈25 ns for `make-struct` (7×). Before the `%make-persistent` optimisation, construction cost ≈877 ns—≈24× over mutable `make-instance` and ≈35× over a struct constructor. The optimisation bypasses `initialize-instance` by calling `allocate-instance` and writing slots via `slot-value` directly; the 4.8× speedup (877 ns → 183 ns) confirms that standard MOP initarg iteration dominates construction cost when there are no aliases. The 19.2× total in Table 8 is a whole-program ratio against the `defstruct+typecase` CL implementation; construction ratios in this section use mutable `make-instance` as the baseline. **Reads** are 1.4–2.1× native; and **Updates** are 33–447× slower (Table 11). Table 12 evaluates candidate thresholds, showing $T = 8$ minimizes update cost for larger objects while keeping common cases on the native path.

Table 11: Actual slot update overhead (µs/op) vs. native CLOS

Slots	Persistent	Mutable	Ratio	Notes
2	1.14 μs	34.6 ns	33×	2 native slots
8	2.34 μs	41.6 ns	56×	8 native slots
32	6.85 μs	28.5 ns	240×	all-native (T=32 config)
48	8.50 μs	41.6 ns	204×[a]	32N + 16-slot trie
128	14.41 μs	32.2 ns	447×	32N + 96-slot trie

[a] Ratio drops vs. 32 slots because the mutable CLOS baseline is 46% slower at 48 slots (discontiguous memory layout), not because persistent overhead decreases.

Table 12: Modeled update cost (μs/op) at candidate thresholds T

N	T=8	T=12	T=16	T=24	T=32	T=8 strategy
8	**0.42**	0.42	0.43	0.42	0.42	all-native
12	0.75	**0.62**	0.62	0.61	0.60	8N + 4V
16	0.94	0.91	0.83	0.78	**0.77**	8N + 8V
24	**1.15**	1.23	1.28	**1.15**	**1.15**	8N + 16V
32	**1.39**	1.47	1.51	1.58	1.52	8N + 24V
48	**1.95**	1.99	2.04	2.12	2.22	8N + 40V

Bold = minimum in row. "kN + jV" = copy k native slots then batch-build j-element overflow vector. All-native = copy N slots directly. Simulated copy costs only; actual `update-slots` is 1.6–3× higher due to MOP iteration overhead (see Table 11).

4.6.1 Predicate Dispatch Micro-benchmark. To evaluate the overhead of FOL's $O(N)$ predicate dispatch, we measured the per-invocation cost of a generic function with 20 methods, each specialized on a distinct Level-3 predicate (e.g., `(x (P0?))`). This configuration exercises the worst-case linear scan and runtime predicate evaluation. We compared FOL's open dispatch (`defgeneric`) and closed dispatch (`defn`) against two Common Lisp baselines: standard CLOS type dispatch (Level-2 equivalent) and a manually-authored `cl:cond` cascade. Table 13 summarizes the results on SBCL 2.6.0 (x86-64).

Table 13: Predicate Dispatch Performance (N=20 clauses).

Dispatch Method	Time/op	Ratio (vs. CLOS)
Standard CLOS (Type)	37.1 ns	1.00×
Manual `cl:cond` Cascade	58.5 ns	1.58×
FOL `defn` (Closed Dispatch)	57.2 ns	1.54×
FOL `defgeneric` (Open Dispatch)	60.6 ns	1.63×

Results show FOL's predicate dispatch adds ≈1.6× overhead relative to optimized CLOS type dispatch. These measurements isolate routing overhead; in expensive predicate workloads, per-invocation cost is dominated by evaluation. The parity between FOL variants and the manual `cl:cond` baseline confirms that FOL's routing is as efficient as hand-written code, bounded only by the lack of $O(1)$ cacheability for Level-3 methods.

4.6.2 Lazy Evolution Micro-Benchmark. To quantify the cost of FOL's lazy schema migration machinery (Section 3.2), we ran two benchmarks against standard-CLOS baselines. Both systems use SBCL's built-in `update-instance-for-redefined-class` lazy migration; the difference is that FOL's recovery of renamed slots is automatic (derived from `:alias` annotations on the class definition), whereas the CL baseline requires a manually written `:after` method that must be updated every time a slot is renamed.

Single-hop (Schema 1 → Schema 2). A 6-slot `<device>` (all native, $T \leq 8$) is redefined to a 12-slot layout crossing the overflow boundary, renaming two slots (`z→z-val`, `w→w-val`). Table 14 compares first-touch migration cost and post-migration steady-state for FOL (`update-slots`, functional) and CL (`setf slot-value`, mutable), with 10,000 dormant instances.

Table 14: Single-hop lazy migration: FOL vs. CL (µs/op, 10K instances)

Phase	FOL	CL	Notes
Migration (first touch)	14.04	7.33	one-time per dormant instance
of which: overhead vs. steady-state	10.75	7.31	SBCL migration + alias recovery
Steady-state update	3.30	0.02	post-migration, repeated use

FOL reifies the distinction between *identity* and *value*. Read access to values is always lock-free, as threads observe stable snapshots; coordinated state mutation is handled via Atom (CAS-based) and Ref (STM) primitives.

5 Threats to Validity

Several threats to validity should be considered. **Single implementation**: All measurements use SBCL 2.6.0 on a single platform; results may differ on other CL implementations or architectures. **Benchmark coverage**: Our evaluation includes both *feature benchmarks* (structural diff, spec-exec) and *adversarial workloads* (naive qsort). This covers a representative range of FOL's performance

Table 15: Multi-hop lazy migration first-touch cost (µs/op, 5K instances per cohort)

Hop depth	FOL	CL	Notes
1-hop (v2→v3)	8.13	4.10	FOL allocates new object; CL mutates in-place
2-hop (v1→v3)	8.36	5.62	extra plist scan (CL); extra hash-table pass (FOL)
3-hop (v0→v3)	7.80	5.13	both dominated by SBCL migration machinery
Steady-state	1.40	0.01	post-migration; uniform across all cohorts

envelope. Benchmarks report means of multiple runs to amortize warmup effects; small LSim configurations (8bit-100 through 8x32-900) were repeated 20 times and medium configurations 3 times. The 32×32×32-30000 circuit was timed once per implementation; the FOL variant required ≈13,800 s wall time.

The `persistent-class` metaclass relies on the MOP hooks listed in Table 1, whose availability and semantics vary across CL implementations; portability to non-SBCL runtimes (e.g., CCL, ECL, ABCL) is untested.

FOL is transpiled to Common Lisp, introducing a threat to dispatch: Level-3 predicate specializers cannot be cached by class signature and require runtime evaluation. Level-2 type checks are evaluated linearly in the current transpiler rather than via a discriminating function, but a native compiler could replace the $O(N)$ scan with a cached type-check sequence for Level-2 and an ordered sequence for Level-3; reported dispatch ratios are therefore upper bounds for what a native implementation could achieve. No static analysis exists—type errors and unreachable clauses are detected at runtime only.

Definition-order tie-breaking is a *modularity hazard*, but its scope is strictly confined. Patterns are incomparable under $\prec$—and therefore subject to load-order sensitivity—only when every position carries the same Level-3 predicate category with no type relationship, or when patterns are structurally incomparable (map vs. sequential). For example, two methods dispatching on `[(x (prime?))]` and `[(x (fibonacci?))]` are incomparable under $\prec$; for a prime Fibonacci argument, the first-loaded method fires. The recommended mitigation is to collapse overlapping Level-3 patterns into a single clause with explicit conditional logic, or to use `:around` to explicitly sequence them; demoting one pattern to a type-level check (Level 2) restores determinism only when semantic generalization is acceptable. Methods that differ in type, arity, or key set are always ordered by $\prec$ without recourse to *idx*, so the hazard cannot arise for the patterns most commonly encountered in practice. A strict approach signaling ambiguity at load time would eliminate the hazard entirely but would deviate from Lisp's last-definition-wins workflow. In multi-library ASDF systems, load order is determined by the system dependency graph; two independently-developed libraries contributing incomparable Level-3 methods to the same generic should document their intended ordering or provide an explicit `:around` bridge method at the library boundary.

6 Related Work

FOL occupies a middle ground between CLOS and purely functional languages. Our trie implementation follows Bagwell [2] and Clojure [6]. Functional lenses [10] provide nested updates in pure languages through a composition algebra; FOL's MOP approach achieves similar ergonomics through established Lisp slot-access interception without requiring a new abstraction layer.

Predicate dispatch was pioneered in Cecil [3] and formalized as a unified theory by Ernst et al. [5]; both require theorem-prover subsumption for method ordering. FOL sacrifices exactness for syntactic predictability. Filtered dispatch [13] and singleton specializers in Dylan [1] also influenced our model. Slate [14] extended multiple dispatch with prototype-based receivers; FOL instead preserves CLOS method combinations. Clojure's `defmulti` [6] dispatches on the return value of a user-supplied function; method ordering is managed via `prefer-method` (see Section 3 for a detailed comparison). FOL adopts the "objects as maps" philosophy of Self [15] but on a functional substrate, and the epochal time model of Clojure [7].

7 Conclusion

We presented two contributions to the design of Lisp object systems. First, a **persistent metaclass system** built on the CLOS MOP: hybrid native/trie storage keeps ≈80% of classes on the 1.4–2.1× read-overhead native path; lazy alias-based schema migration delivers transparent instance evolution across class redefinitions; and the `%make-persistent` construction path reduces object creation to 5× over mutable `make-instance`. Second, a **specificity-ordered predicate dispatch** system whose syntactic total order $\prec_{\text{disp}}$ resolves method selection without a theorem prover, covering all structurally or type-differentiated patterns automatically and confining load-order sensitivity to incomparable Level-3 predicates; this provides a predictable alternative to theorem-prover-based subsumption.

Both contributions are validated by the benchmarks in Section 4. Isolated micro-benchmarks reach 33–447× for persistent slot updates and 113.7× for feature-level operations; the end-to-end LSim event-queue simulation reaches 6.9× at 1.18 billion gate evaluations, where the fixed $O(\log n)$ per-merge allocation cost is amortized over growing simulation work per queue operation. The AST optimizer isolates the cost structure: at most ≈1.6× from dispatch routing (upper bound; AST uses mixed-level patterns that short-circuit earlier) and ≈12× from persistent field access, with $O(1)$ speculative rollback as a structural benefit.

Open problems include: native compilation to replace the $O(N)$ predicate scan with a static type-check sequence, static clause analysis to detect unreachable patterns at load time (Level-2 overlaps are decidable via the class precedence list; Level-3 overlap detection would require programmer-supplied subsumption annotations), and distribution of persistent value identities across nodes for coordinated concurrent state.

The implementation is available at https://github.com/frankadrian314159/fol.

References

[1] Apple Computer, Inc. 1992. *Dylan - An Object-Oriented Dynamic Programming Language.* Apple Computer, Inc.

[2] Phil Bagwell. 2001. *Ideal Hash Trees*. Technical Report 1. EPFL, Lausanne, Switzerland.
[3] Craig Chambers. 1993. The Cecil Language: Design and Performance. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications*. 243–257.
[4] Clark Allen Crane. 1972. *Linear Lists and Priority Queues as Balanced Binary Trees*. Technical Report STAN-CS-72-259. Stanford University.
[5] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP'98—Object-Oriented Programming*. Springer, 186–211.
[6] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. 1.
[7] Rich Hickey. 2009. Are We There Yet? JVM Languages Summit Keynote. https://github.com/matthiasn/talk-transcripts/blob/master/Hickey_Rich/AreWeThereYet.md
[8] Rich Hickey. 2014. Transducers. Clojure Blog. https://clojure.org/reference/transducers
[9] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT press.
[10] Edward Kmett. 2012. Lenses, Folds, and Traversals. In *Workshop on Generic Programming*.
[11] Michele Lanza and Radu Marinescu. 2006. *Object-Oriented Metrics in Practice*. Springer.
[12] Chris Okasaki. 1998. *Purely Functional Data Structures*. Cambridge University Press.
[13] Doug Orleans. 2002. Filtered Dispatch. In *Proceedings of the 17th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 20–26.
[14] Lee Salzman and Jonathan Aldrich. 2005. Prototypes with Multiple Dispatch: An Expressive and Dynamic Object Model. In *ECOOP 2005–Object-Oriented Programming*. Springer, 312–336.
[15] David Ungar and Randall B Smith. 1987. Self: The power of simplicity. In *ACM SIGPLAN Notices*, Vol. 22. ACM, 227–242.